Mastering Docker: A Comprehensive Guide to Essential Commands

Docker has become an indispensable tool in modern software development, enabling developers to build, ship, and run applications in isolated environments called containers. This guide provides a detailed overview of essential Docker commands, complete with explanations and practical examples, to help you effectively manage your containerized applications.

1. Managing Docker Images

Docker images are the building blocks of containers. They are lightweight, standalone, and executable packages that include everything needed to run an application.

Command	Description	Example
`docker build`	Builds a Docker image from a Dockerfile.	`docker build -t my-app:1.0 .`
`docker images`	Lists all the Docker images available on your local system.	`docker images`
`docker pull`	Downloads a Docker image from a container registry like Docker Hub.	`docker pull nginx:latest`
`docker push`	Uploads an image to a container registry.	`docker push your-username/my-app:1.0`
`docker rmi`	Removes one or more Docker images.	`docker rmi my-app:1.0`
`docker tag`	Creates a new tag for an existing image.	`docker tag my-app:1.0 your-username/my-app:1.0`
`docker history`	Shows the history of an image.	`docker history nginx:latest`
`docker save`	Saves one or more images to a tar archive.	`docker save -o my images.tar nginx:latest ubuntu:latest`
`docker load`	Loads an image from a tar archive or STDIN.	`docker load -i my_images.tar`

2. Running and Managing Docker Containers

A Docker container is a runtime instance of a Docker image. The `docker run` command is the primary way to create and start a new container.

Command	Description	Example
`docker run`	Creates and starts a new container from an image.	`docker run -d --name my-web-server -p 8080:80 nginx`
`docker ps`	Lists all running containers.	`docker ps`
`docker ps -a`	Lists all containers, including stopped ones.	`docker ps -a`
`docker start`	Starts one or more stopped containers.	`docker start my-web-server`
`docker stop`	Stops one or more running containers.	`docker stop my-web-server`
`docker restart`	Restarts a container.	`docker restart my-web-server`
`docker rm`	Removes one or more stopped containers.	`docker rm my-web-server`
`docker kill`	Forcefully stops and removes a running container.	`docker kill my-web-server`
`docker exec`	Executes a command inside a running container.	`docker exec -it my-web-server /bin/bash`
`docker logs`	Fetches the logs of a container.	`docker logs my-web-server`
`docker inspect`	Displays detailed information on one or more containers.	`docker inspect my-web-server`
`docker attach`	Attaches your terminal's standard input, output, and error to a running container.	`docker attach my-web-server`
`docker cp`	Copies files or folders between a container and the local filesystem.	`docker cp my-web-server:/usr/share/nginx/html/index.html .`
`docker commit`	Creates a new image from a container's changes.	`docker commit my-web-server my-custom-nginx:latest`

The `docker run` Command in Detail

The `docker run` command is highly versatile with numerous options to customize container behavior. Here are some of the most common options:

- `-d`: Runs the container in detached mode (in the background).
- `--name`: Assigns a custom name to the container.
- `-p`: Publishes a container's port(s) to the host. For example, `-p 8080:80` maps port 8080 on the host to port 80 in the container.
- `-v`: Mounts a host directory or a named volume into the container.
- `-e`: Sets environment variables inside the container.
- `--rm`: Automatically removes the container when it exits.
- `-it`: Creates an interactive terminal for the container.

3. Docker Networking

Docker provides networking capabilities to connect containers with each other and the outside world.

Command	Description	Example
`docker network ls`	Lists all the networks.	`docker network ls`
`docker network create`	Creates a new network.	`docker network create my-custom-network`
`docker network connect`	Connects a container to a network.	`docker network connect my-custom-network my-web-server`
`docker network disconnect`	Disconnects a container from a network.	`docker network disconnect my-custom-network my-web-server`
`docker network inspect`	Displays detailed information on one or more networks.	`docker network inspect my-custom-network`
`docker network rm`	Removes one or more networks.	`docker network rm my-custom-network`

4. Docker Data Management (Volumes)

Volumes are the preferred mechanism for persisting data generated by and used by Docker containers.

Command	Description	Example
`docker volume ls`	Lists all the volumes.	`docker volume ls`
`docker volume create`	Creates a new volume.	`docker volume create my-data-volume`
`docker volume inspect`	Displays detailed information on one or more volumes.	`docker volume inspect my-data-volume`
`docker volume rm`	Removes one or more volumes.	`docker volume rm my-data-volume`
`docker volume prune`	Removes all unused local volumes.	`docker volume prune`

5. System and Cleanup Commands

These commands help in managing the Docker system and cleaning up unused resources.

Command	Description	Example
`docker system df`	Shows Docker disk usage.	`docker system df`
`docker system prune`	Removes all unused containers, networks, images (both dangling and unreferenced), and optionally, volumes.	`docker system prune -a --volumes`
`docker info`	Displays system-wide information about Docker.	`docker info`
`docker version`	Shows the Docker version information.	`docker version`

This comprehensive list provides a strong foundation for working with Docker. For a more in-depth look at each command and its options, refer to the official Docker documentation.

---

create docker network

docker network create mongo-network

start mongodb

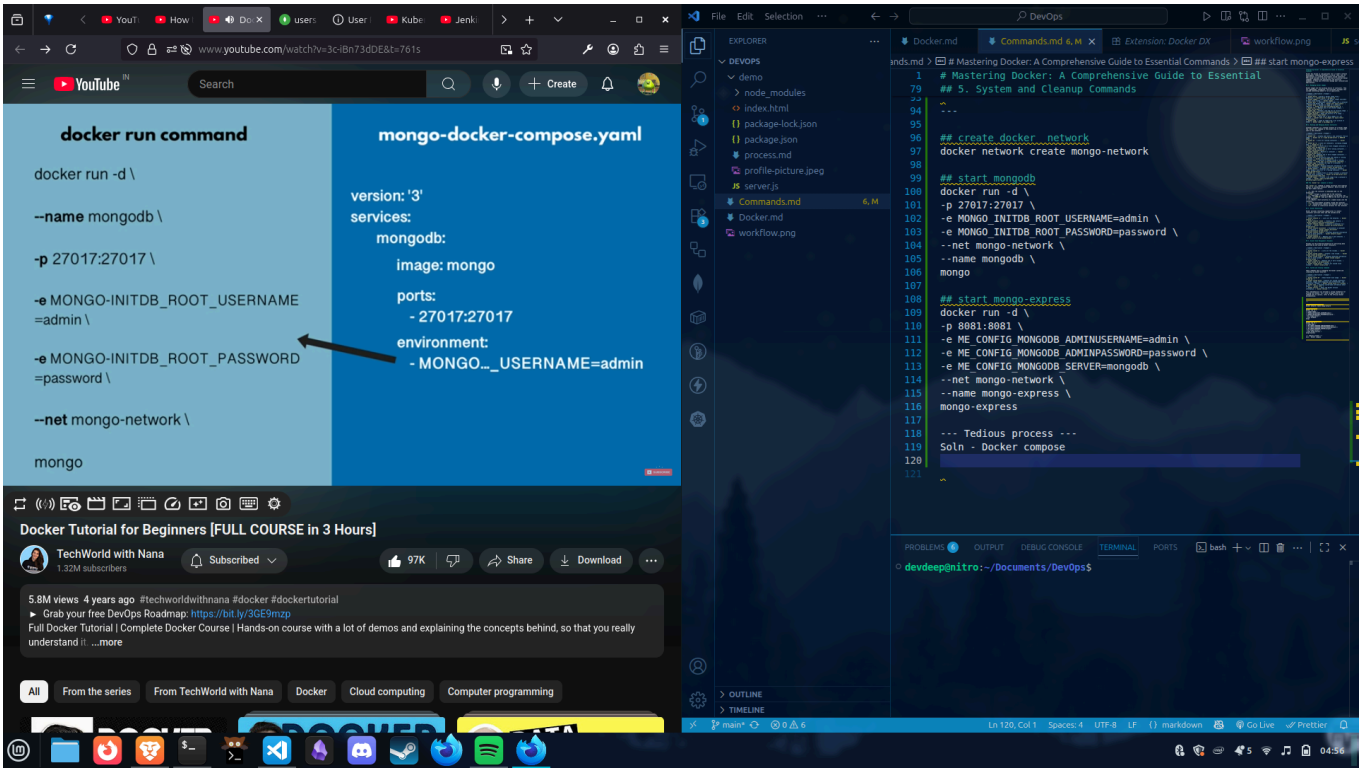
docker run -d
-p 27017:27017

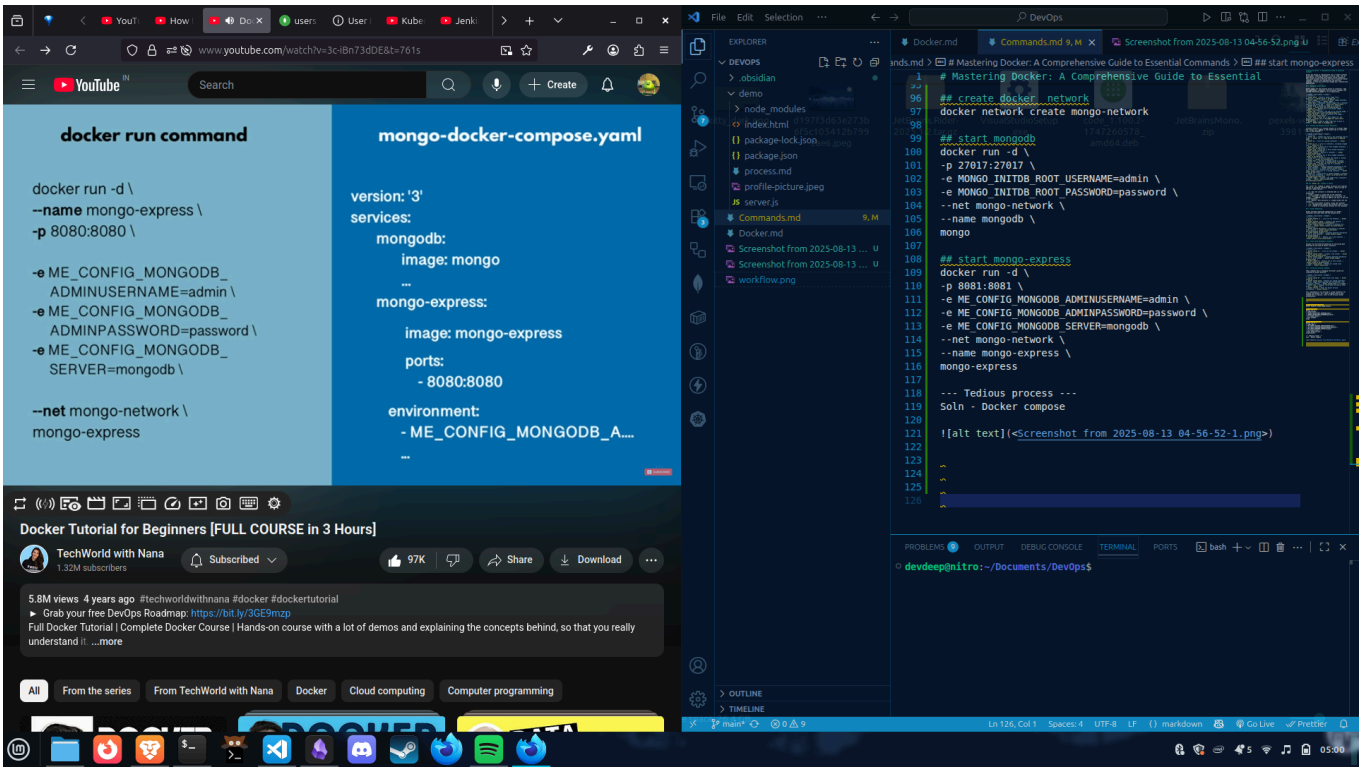
-e MONGO_INITDB_ROOT_USERNAME=admin
-e MONGO_INITDB_ROOT_PASSWORD=password
--net mongo-network
--name mongodb
mongo

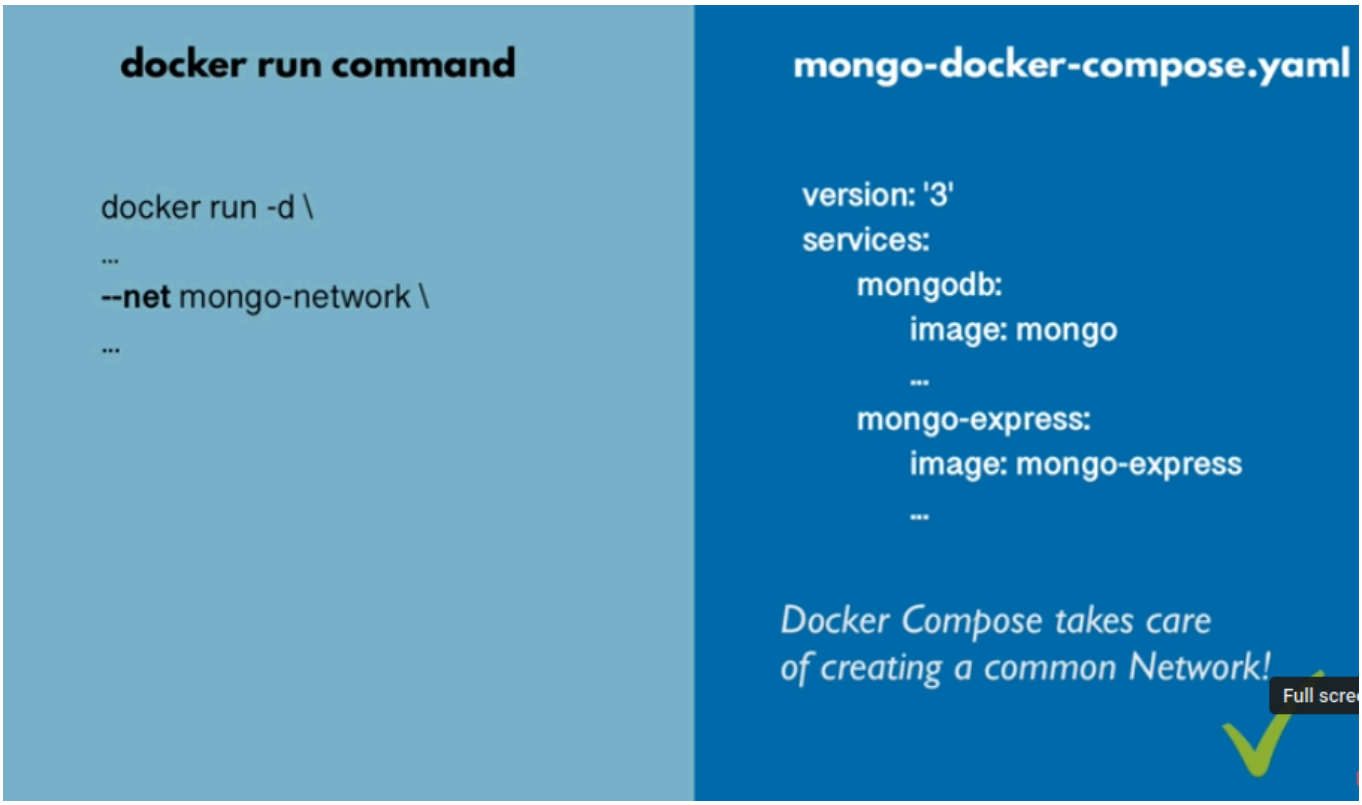
start mongo-express

docker run -d
-p 8081:8081
-e ME_CONFIG_MONGODB_ADMINUSERNAME=admin
-e ME_CONFIG_MONGODB_ADMINPASSWORD=password
-e ME_CONFIG_MONGODB_SERVER=mongodb
--net mongo-network
--name mongo-express
mongo-express

--- Tedious process ---
Soln - Docker compose







Docker Compose

Main Commands

These are the most frequently used commands for managing your application stack.

Command	Description
`up`	Builds, (re)creates, starts, and attaches to containers for a service. If the containers are already running, it will stop and recreate them to pick up any changes in the `docker-compose.yml` file or a newly built image. Use the `-d` flag to run containers in detached mode (in the background).
`down`	Stops and removes containers, networks, and the default network created by `up`. You can use the `-v` flag to also remove named volumes.
`ps`	Lists the containers for the services in your project, including their status, ports, and names.
`logs`	Displays log output from services. You can follow the logs in real-time using the `-f` flag.
`build`	Builds or rebuilds the Docker images for your services.
`pull`	Pulls the service images from a registry.
`restart`	Restarts all stopped and running services.
`stop`	Stops running containers without removing them. They can be started again with `start`.
`start`	Starts existing containers for a service.

Application and Service Management

These commands provide more granular control over your services.

Command	Description
`run`	Runs a one-time command against a service. For example, `docker-compose run web python manage.py migrate`.
`exec`	Executes a command in a running container.
`pause`	Pauses services.
`unpause`	Unpauses services.
`kill`	Forces running containers to stop by sending a `SIGKILL` signal.
`rm`	Removes stopped service containers.
`port`	Prints the public port for a port binding.
`events`	Streams container events for the services in the project.

Configuration and Image Management

These commands help you manage your Compose file and images.

Command	Description
`config`	Validates and views the final configuration after processing the Compose file, including resolving variables and merging multiple files.
`images`	Lists the images used by the services.
`push`	Pushes service images to a registry.

Resource Management

These commands are used for managing networks and volumes.

Command	Description
`network`	A parent command for managing networks with subcommands like `create`, `rm`, `ls`, `inspect`, `connect`, and `disconnect`.
`volume`	A parent command for managing volumes with subcommands like `create`, `rm`, `ls`, and `inspect`.

Other Useful Commands

Command	Description
`cp`	Copies files or folders between a service container and the local filesystem.
`create`	Creates containers for a service. They can be started later with `start`.
`top`	Displays the running processes for a service.
`version`	Displays Docker Compose version information.
`help` or `--help`	Provides help and usage information for a command.

Global Options

These options can be used with most Docker Compose commands to modify their behavior.

Option	Description
`-f, --file`	Specify an alternate Compose file (or a list of files).
`-p, --project-name`	Specify an alternate project name (the default is the directory name).
`--project-directory`	Specify an alternate working directory.

Option	Description
`--env-file`	Specify an alternate environment file.
`--profile`	Specify a profile to enable.

This list provides a comprehensive overview of the commands available in Docker Compose. For more detailed information on a specific command and its options, you can always run `docker-compose <command> --help`.
